# Diagrams and design decisions

This document explains every diagram for PrintXpress and the choices behind it. It covers Task B of the assignment, the system and database design. The diagram source files are in `diagrams/` as draw.io files.

## How to open the diagrams

1. Go to diagrams.net or open the draw.io desktop app, or use the draw.io extension inside VS Code.
2. Open any `.drawio` file from the `diagrams/` folder.
3. To put a diagram in a report, use File, then Export as, then PNG or PDF.

All diagrams use one style: the brand accent for borders, text, and lines, a light accent fill for shapes, and right-angle connectors with no curves. This keeps them readable in print and on screen.

## Use case diagram

File: `diagrams/01-use-case.drawio`.

The use case diagram shows what a customer can do with PrintXpress. There is one human actor, the customer, because this version is a customer-only app with no separate staff app. The system boundary box holds every action the app offers.

Design decisions:

1. One actor keeps the scope honest. The shop side is out of scope, so adding a staff actor would imply features that are not built.

2. Two use cases are modelled as included steps rather than standalone actions. Place an order includes upload design or enter text and includes choose pickup or delivery, because those steps always happen as part of placing an order. The include arrows show this.

3. Reschedule or cancel order extends track order status. The extend arrow says the action is optional and only happens in some cases, here only while the order is still processing.

4. The remaining use cases connect straight to the customer, since the customer starts each of them.

## Class diagram

---

File: `diagrams/02-class.drawio`.

The class diagram shows the main domain classes, their fields, their key methods, and how they relate. The classes match the database tables, so the design stays consistent from object to storage.

Design decisions:

1. User is the centre of the model. A user owns addresses, orders, saved designs, and notifications. These are drawn as aggregation, the open diamond, because the parts belong to the user but are still their own objects.

2. Order to order item is composition, the filled diamond. An order item has no meaning without its order, so it lives and dies with the order. This maps to the cascade delete in the database.

3. Multiplicities are shown at both ends. For example a user has zero or more orders (1 to 0..*), and an order has one or more items (1 to 1..*).

4. Product to order item is a plain association. A product is referenced by many order lines but is not owned by them, so deleting an order line must never delete the product.

5. Methods are kept short and practical, for example `Order.place()`, `Order.cancel()`, and `Promotion.isActive()`. They name the behaviour without locking in implementation detail.

## Activity diagram

---

File: `diagrams/03-activity.drawio`.

The activity diagram models the core flow, placing an order, from opening the app to the confirmation. It is the most important journey in the app, so it is worth modelling end to end.

Design decisions:

1. The first decision checks if the customer is logged in. If not, they log in or register, then both paths merge and carry on. This avoids drawing the browse flow twice.

2. The second decision lets the customer either upload a design file or type custom text. Both branches merge back into choosing pickup or delivery, since the order needs one or the other.

3. The third decision handles delivery. Only a delivery order asks for an address, so the address step sits on the yes branch and the pickup path skips it.

4. Saving the order and showing the confirmation are separate actions, because the save writes to the database and the confirmation is what the customer sees. Keeping them apart makes the data step explicit.

5. Every connector turns at a right angle and no two unrelated paths cross, so the flow is easy to follow.

# Entity relationship diagram

File: `diagrams/04-er.drawio`.

The ER diagram shows the database tables, their columns, the primary keys, the foreign keys, and the relationships in crow's foot notation. It is the storage view of the class diagram.

Design decisions:

1. Every table has a single auto-generated primary key of type Long. This is simple, fast, and works well with Room.
2. Foreign keys link children to parents. For example `order_items.orderId` points to `orders`, and `products.categoryId` points to `categories`.
3. The design is normalised to third normal form. Each fact is stored once. An order stores its total once it is placed, which is a deliberate choice so a past order keeps its price even if the catalog changes later.
4. Delivery address and promotion are optional foreign keys on the order, since a pickup order has no address and an order may have no promotion. These are shown as optional relationships.
5. Product option lists, such as sizes and materials, are stored as short comma separated text on the product. This keeps the schema small for a beginner. If the project grows, these could move to their own tables.

The full table breakdown, with every column and type, is in `04-database-design.md`.

## Sequence diagram

File: `diagrams/05-sequence.drawio`.

The sequence diagram shows the messages that pass between parts of the app when a customer places an order. It reads top to bottom in time, left to right across the layers.

The five lifelines are the customer, the order screen (Compose UI), the order view model, the order repository, and the Room database. This matches the app's structure, where the UI talks to a view model, the view model talks to a repository, and the repository talks to Room.

Design decisions:

1. The UI never touches the database directly. It calls the view model, which calls the repository, which calls Room. This keeps the layers clean and easy to test.

2. Solid arrows are calls and dashed arrows are returns. The new order id flows back from Room through the repository and the view model to the UI.

3. The final return shows the UI telling the customer the order is placed, which lines up with the last steps of the activity diagram.

4. This separation is the reason the app uses a repository at all. It is a small extra step that makes the data flow obvious and keeps each class focused.